

Simplified Pipelined RISC-V Processor

Introduction

The goal of this project is to design a simplified pipelined RISC-V microprocessor that was introduced in the chapter 7.5 of Harris and Harris textbook [1]. For convenience, datapath diagram and main decoder table are reproduced below in Figures 1-2.

Project assignment

Your task is to implement datapath and control modules to support execution of *add*, *addi*, *sub*, *and*, *andi*, *or*, *ori*, *slt*, *slti*, *lw*, *sw*, *beq*, *jal*, *lui* instructions. Note that *jal* and *lui* are not implemented in the textbook design so you have to extend Figures 1, 2 to implement these new instructions.

The skeletons for datapath and control modules as well as all other files are provided in the starter package. The hierarchy is similar to the lab 4 (single cycle processor) of 154A. Refer to Figure 3 for how the processor is partitioned into modules.

Be sure to use the parameters defined in “ucsbece154a_defines.vh” wherever possible to avoid magic numbers. Correlate signal names in the Verilog code with the wire names on the schematic.

Testing your design

A testbench file is provided. An instruction memory (TEXT segment) starts at 0x10000 address and data memory starts at 0x01000000 in accordance to Linux memory map. The code is loaded with “text.dat” that represents assembly code in Figure 4. Note that the code does not test all possible cases and you may want to write your own custom code for a more comprehensive verification.

Hints

- Refer to the ISA appendix from Ref. [1] or open source documentation [2] for RISC-V instruction formats
- Start first with extending the diagram and decode tables
- You may reuse RTL designs from 154A lab and/or textbook. For example, textbook provides implementation of the hazard unit. You may reuse any code from ECE154A lab4 and lab5 solutions that are also provided in the startup package.
- New instructions can be implemented with minimal changes (by extending few muxes, adding a mux, and pipelining more control signals)
- Do not forget resetting all registers at the start

Grading and deadlines

- **Part A [20 points]**. Lab check off, due **April 11th** in CSIL lab
- **Part B [60 points]**. RTL code, with grade assigned by Gradescope autograding feature, due **April 11th**

- **Part C [20 points].** Lab report due **April 13th**

What to Turn In

1. Submit your RTL files to “lab1 RTL” folder on Gradescope
2. Submit a single PDF file to “lab1 report” folder on Gradescope. The report should contain the following items:
 - a. Marked up versions of the datapath schematic (Figure 2) and decoder table (Figure 1) that add the *jal* and *lui* instructions.
 - b. Screenshots of the entire waveform with the following signals (in the given order): CLK, Reset, PCF, InstD, RD1E, RD2E, ExtImmE, ALUResultM, WriteDataM, ResultW for three provided testbench files. Display all signals in hexadecimal and make sure they are readable.
 - c. Indicate how many hours you spent on this lab.

References:

- [1] S. Harris and D. Harris, Digital Design and Computer Architecture, RISC-V Edition: RISC-V Edition 1st Edition, ISBN-10: 0128200642, 2021
- [2] Volume 1, Unprivileged Specification version 20191213, <https://riscv.org/technical/specifications/>

op	Instruct.	RegWrite	ImmSrc	ALUSrc	MemWrite	ResultSrc	Branch	ALUOp	Jump
3	lw	1	00	1	0	01	0	00	
35	sw	0	01	1	1	XX	0	00	
51	R-type	1	XX	0	0	00	0	10	
99	beq	0	10	0	0	XX	1	01	
19	I-typeALU	1	00	1	0	00	0	10	
	jal								
	lui								

Fig. 1. Extended functionality for the main decoder. Fill in missing values in the table

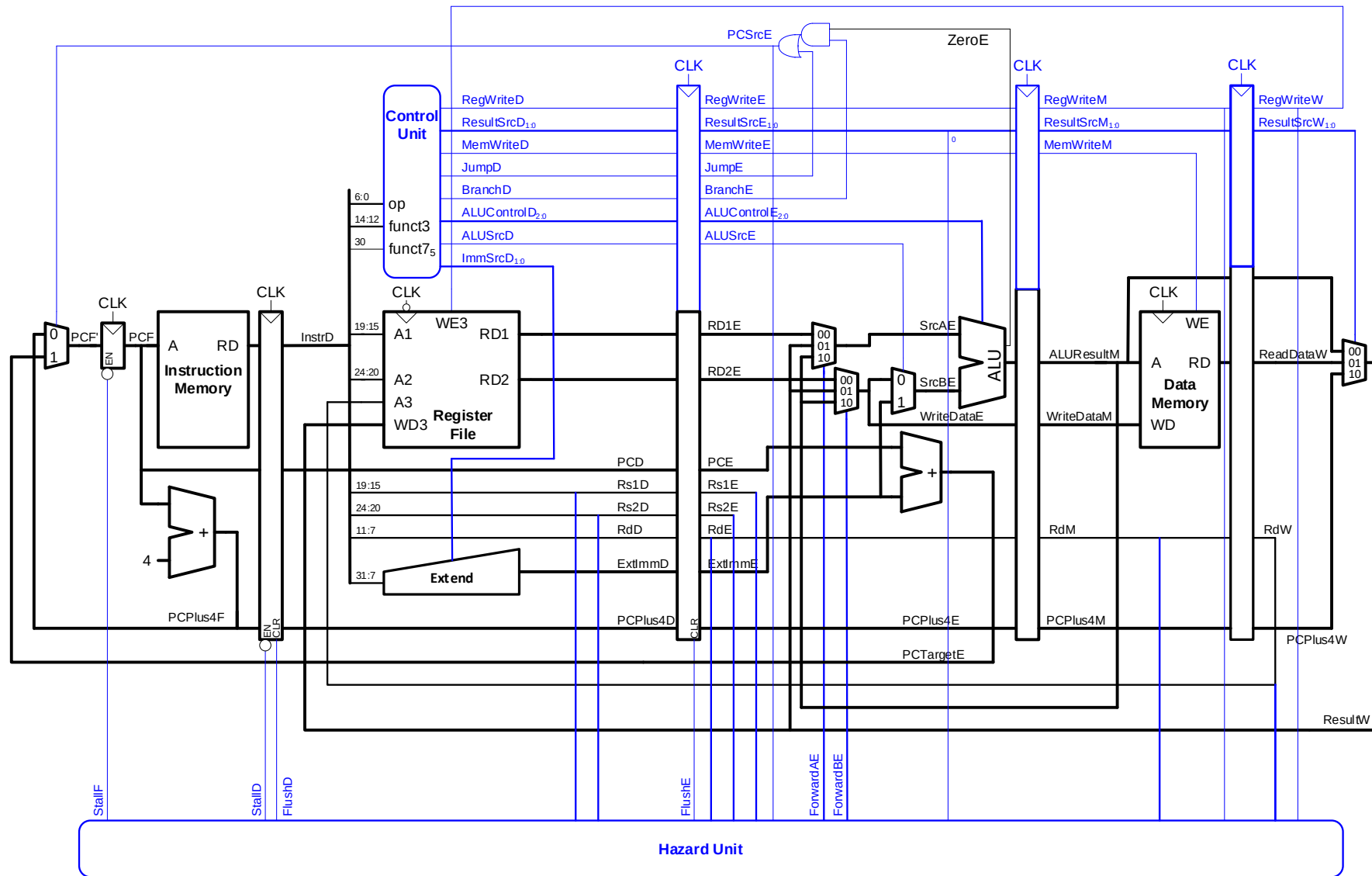


Fig. 2. Datapath and control for pipelined RISC-V processor. Add new components to implement *jal* and *lui*

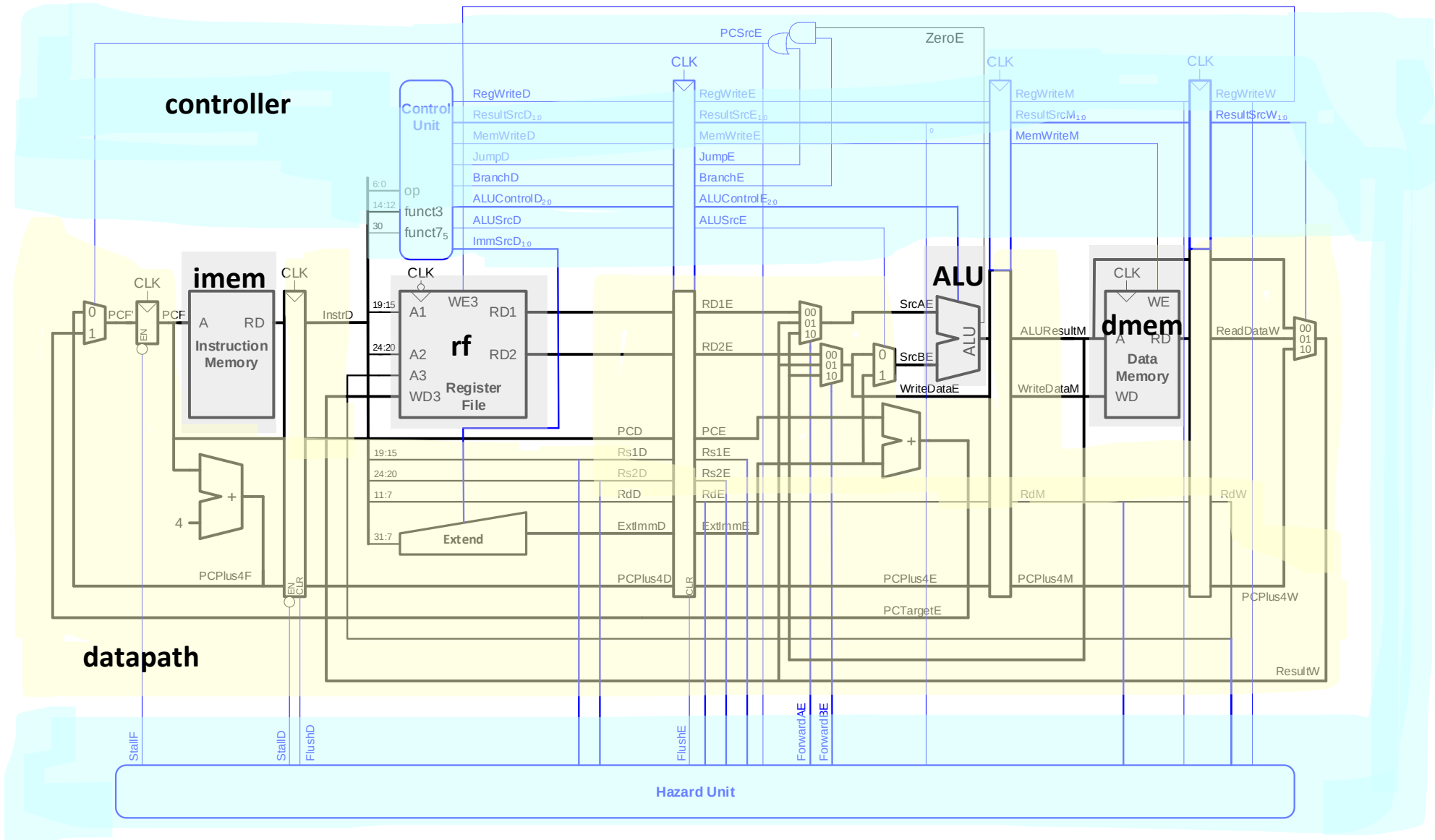


Fig. 3. Design hierarchy. “riscv_pipe” module connects datapath and controller modules. The complete design is instantiated in the “top” module .

```

.data

.text
.globl  main
main:   addi    x2, x0, 5
        addi    x3, x0, 12
        addi    x7, x3, -9
        or      x4, x7, x2
        and     x5, x3, x4
        add     x5, x5, x4
        beq     x5, x7, end
        slt     x4, x3, x4
        beq     x4, x0, around
        addi    x5, x0, 0
around: slt     x4, x7, x2
        add     x7, x4, x5
        sub     x7, x7, x2
        lui     x8, 0x10000
        add     x3, x3, x8
        sw      x7, 84(x3)
        lw      x2, 96(x3)
        add     x9, x2, x5
        jal     x3, end
        addi    x2, x0, 1
end:    add     x2, x2, x9
        add     x3, x3, x8
        lui     x1, 0x10
        sub     x3, x3, x1
        sw      x2, 0x20(x3)
        lui     x2, 0x0BEEF
        sw      x2, 0x24(x3)

done:   beq     x2, x2, done

```

Figure 4. Testbench assembly code (same as in lab 5 of ECE154A)